

SQL Audit Pipeline

Deep Analysis Report

Database:AdventureWorks2022 | Server: 192.168.75.50,1433 | Audit: 07/09/2026

This report contains the complete deep analysis of the SQL audit pipeline output — covering every tool's findings, a head-to-head comparison between the Combo (SQLFluff + TSQLAnalyzer + Microsoft SQL Assessment) and SQLEnlight, a detailed explanation of what each tool does and how it works, the qualities that justify using each tool, and a clarification on SQLEnlight's DBCC behavior.

Executive Scores at a Glance

SQLFluff violations 7	TSQLAnalyzer issues 384	MS SQL findings (med) 13
SQLEnlight violations 850	Tech debt (hours) 214	Objects with issues 100%

Tool 1 — SQLFluff

SQLFluff is the code style police of your pipeline. It reads your SQL like a spell-checker reads an essay — it doesn't care if the logic is correct, only if the formatting follows the rules you defined. Before it even runs, it has to extract the database and convert it to plain SQL — that's the 4-step process at the top of the output.

What the 4-step pipeline did

1	Extract database → .dacpac Connected to SQL Server and exported the entire database schema into a .dacpac file (a zipped package of all your tables, views, procs, etc.) using Microsoft's SqlPackage tool. Took 5 seconds.
2	Generate SQL script from .dacpac Converted the .dacpac into a flat .sql file — a single giant script that recreates the entire database from scratch. This is what SQLFluff actually reads. Took 8 seconds.
3	SQLFluff fix — lint and auto-correct SQLFluff read the .sql file and attempted to auto-fix violations. It found issues but could only fix some of them. 7 violations were unfixable — they require a human to change the code manually.
4	Deploy fixed SQL back to database Applied the fixed script back to the database. "Mise à jour terminée" = update completed successfully.

What SQLFluff actually found

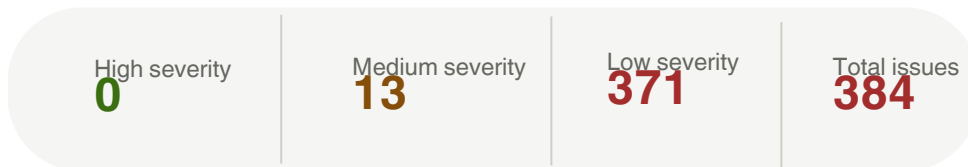
<i>TQ03</i>	Empty GO batch at line 9. The script had a lone GO statement with nothing around it — dead whitespace noise in T-SQL scripts. Not harmful, but messy.	Low
<i>LT05</i>	Line too long (103 chars vs. 80-char limit) at line 10. Keeping lines short makes code easier to read in an editor or side-by-side diff. 103 characters means you have to scroll horizontally — annoying in code reviews.	Low
<i>CP02</i>	Identifiers not in UPPERCASE — 4 occurrences (lines 16–19). Your config says all unquoted identifiers must be UPPER_CASE. This is about consistency across the team's code.	Low
<i>PRS</i>	Parse error at line 22 — :on error exit syntax. SQLFluff couldn't understand this SQLCMD scripting mode command. It's not pure T-SQL. SQLFluff bailed out — this is why 7 violations were unfixable.	Parse Error

Key takeaway: The SQLCMD ':on error exit' command at line 22 broke the SQLFluff parser. Everything after line 22 was never fully linted. Your actual violation count could be much higher than 7 on a fully lintable file.

Tool 2 — TSQLAnalyzer

TSQLAnalyzer is the code quality auditor. While SQLFluff checks formatting, TSQLAnalyzer checks the logic and structure of your T-SQL code — things that won't crash your database today but will cause bugs, security issues, or performance headaches tomorrow.

The score:



Best practices violations (the important ones)

SRD0701	Query Store not enabled. Query Store is SQL Server's flight recorder — it tracks query execution plans over time so you can see when a query suddenly got slow and roll back to the old plan. A production DB without this is flying blind.	Low
SRD0704	TARGET_RECOVERY_TIME not set. Controls how often SQL Server writes checkpoint data to disk. If not set, recovery after a crash could take much longer. Recommended: 60 seconds.	Low
SRD0047	Column name/type mismatches across schemas (many occurrences). StartDate exists as DATE in HumanResources but DATETIME in Production. Silent type conversion on JOINS can cause wrong results or slow queries.	Low
SRD0020	JOIN without a backing foreign key. The code joins two tables but no FK constraint enforces the relationship is valid. Orphaned records cause the JOIN to silently return wrong data.	Low
SRP0020	Table without a clustered index (HEAP table). Reading requires a full table scan every time — like reading a book with no table of contents. Destroys query performance as the table grows.	Low
SRP0018	Query uses 9–10 JOINS. Every JOIN multiplies the number of rows SQL Server processes. A 10-way JOIN on large tables can grind a server to a halt.	Low
SRP0025	SELECT * inside EXISTS subqueries. Writing IF EXISTS (SELECT * FROM ...) fetches every column. SELECT 1 is enough — it runs dramatically faster.	Low
SRP0004	Trigger returns a result set. Triggers are background events — the calling application doesn't expect them to send data back. Can cause unexpected behavior and driver errors.	Low

SR0014 <i>(MS)</i>	Data loss on DateTime→Date conversion flagged in uspUpdateEmployeeHireInfo, uspUpdateEmployeeLogin, uspUpdateEmployeePersonalInfo. The time part is silently discarded. Microsoft's own analyzer flagged this.	Low
SR0007 <i>(MS)</i>	Nullable columns in predicates. If a column can be NULL and you use it in a WHERE condition, the condition can silently evaluate to NULL — causing rows to be unexpectedly excluded from results.	Low
SR0008 <i>(MS)</i>	@@IDENTITY used in uspLogError. @@IDENTITY returns the last identity inserted anywhere in the session, including inside triggers. Use SCOPE_IDENTITY() instead — scoped to your current statement only.	Low
SRP0016	WHERE clause compares different data types. SQL Server converts one type on-the-fly (implicit conversion) — prevents index usage, forcing a full table scan even if a perfect index exists.	Low
SRP0009	Function wrapped around a column in WHERE clause (non-sargable). E.g., WHERE YEAR(OrderDate) = 2024. SQL Server can't use the index. Rewrite as a range condition instead.	Low

Naming standard violations (SRN0007)

TSQLAnalyzer was configured with company naming conventions — regex patterns like `^PK_Department$` for primary keys and `^UX_Employee_.*` for unique indexes. AdventureWorks uses Microsoft's own conventions (e.g., `PK_Department_DepartmentID`, `AK_Employee_LoginID`), so nearly every primary key and unique index was flagged across all schemas: HumanResources, Person, Production, Purchasing, Sales, dbo. These are cosmetic, not dangerous — but they show the tool is enforcing your org's standards, not Microsoft's defaults.

Tool 2b — Microsoft SQL Assessment

This is the infrastructure and runtime health check. Unlike TSQLAnalyzer which reads source code, the Microsoft SQL Assessment tool connects to the live SQL Server instance and inspects what's actually happening at runtime — index usage, fragmentation, disk space, recovery settings. This is the DevOps perspective, not the developer perspective.

Medium severity findings (the ones that need action)

<i>Unused Indexes</i>	13 unused indexes. AK_Document_rowguid, IX_SalesOrderHeader_CustomerID, IX_Employee_OrganizationNode, and others have never been used since the last statistics reset. Indexes cost write performance — every INSERT/UPDATE/DELETE must update all indexes. Dead indexes are pure overhead.	Medium
<i>DBCC CHECKDB</i>	Run DBCC CHECKDB. This is the database integrity check — it verifies every page, allocation, and relationship. Not running it regularly means corruption could go undetected until it causes data loss. Should be a scheduled weekly job.	Medium
<i>Fragmentation</i>	Index fragmentation detected. dbo.DatabaseLog HEAP: 23.71%. HumanResources.JobCandidate PK: 9.09%. Production.BillOfMaterials: 10–11%. Production.Product PK: 7.69%. Fragmentation means pages are physically scattered — reads require more I/O. Above 10% warrants a rebuild or reorganize job.	Medium
<i>Recovery Model</i>	Recovery model is not FULL or BULK_LOGGED. Database is using SIMPLE recovery — transaction log backups are impossible. If the server crashes, you can only restore to the last full backup. FULL recovery is required for point-in-time recovery in production.	Medium
<i>Disk Space</i>	Only 1% free space on data files. Total: 200 MB. Free: 1.75 MB. Threshold: 25%. If the database runs out of space, all writes fail instantly. The database goes into an error state. Add storage or pre-grow the data file immediately.	Medium

The 1% disk space finding is the most urgent item in this entire audit. A database that runs out of data file space stops accepting writes completely. This needs to be addressed before anything else.

Tool 3 — SQLEnlight

SQLEnlight is the heavyweight semantic analyzer. It connects directly to the live database and analyzes not just the T-SQL source but also the actual execution plans that SQL Server produces when it runs your queries. It caught 850 violations across 53 objects — 100% of analyzed objects had at least one issue.

Top 10 violated rules

SA0274	Missing USE statement — 131 occurrences (15%). Every standalone script should start with USE DatabaseName; to explicitly declare which database it runs against. Without it, running in the wrong context causes data corruption.	131
SA0261	Line length exceeded — 130 occurrences (15%). SQLEnlight found this far more broadly than SQLFluff (130 vs 1 instance) because it analyzed the actual stored procedures and views inside the DB, not just the generated deploy script.	130
SA0122	Nullable columns used in expressions without ISNULL() — 114 occurrences (13%). If a column can be NULL and you use it in a math expression, the entire result becomes NULL. Fix: ISNULL(Price,0) * ISNULL(Quantity,0).	114
SA0044	No index on foreign key columns — 35 occurrences (4%). SQL Server does not automatically create indexes on FK columns. Flagged: Sales.CurrencyRate, Sales.Customer, Production.Product, and more.	35
SA0189	Stored procedure executed without checking result — 20 occurrences (2%). Calls with no IF @@ERROR or TRY/CATCH — silent failures.	20
SA0126	Implicit type conversion — 19 occurrences (2%). SQLEnlight pinpoints it at exact line level: uspUpdateEmployeeHireInfo line 20 — comparing date to datetime. Breaks index usage.	19
EX0018	High-cost Sort operation in execution plan — 16 occurrences (3%). Unique to SQLEnlight — it actually executed queries and read the execution plan. Sort node costing 21% in uspGetEmployeeManagers.	16
SA0119	Table source not aliased — 16 occurrences (3%). Writing FROM HumanResources.Employee instead of using an alias makes column references in complex joins verbose and ambiguous.	16
SA0078	Statements not terminated with semicolons. T-SQL recommends semicolons and certain statements (like CTEs) require a preceding one. Missing semicolons cause parse errors in future SQL Server versions.	Many
SA0042A	Column name contains whitespace — [AWBuildVersion].[Database Version]. A column name with a space must always be quoted with brackets. Easy to forget, causes query failures if brackets are omitted.	1

Combo vs. SQLEnlight — Head-to-Head Comparison

Two different doctors examining the same patient — they'll find some of the same things, but each has different equipment and different areas of expertise.

What the Combo found that SQLEnlight missed

- ✓ Infrastructure issues: disk space (1% free), recovery model (SIMPLE), DBCC CHECKDB reminder
- ✓ Index fragmentation with exact percentages (23.71%, 11.11%, etc.)
- ✓ Unused indexes with runtime usage statistics
- ✓ Database config issues: Query Store disabled, TARGET_RECOVERY_TIME not set
- ✓ SQLCMD dialect parse error — SQLFluff discovered it can't lint SQLCMD scripts
- ✓ Microsoft SR0008: @@IDENTITY risk in uspLogError — specific function-level warning

What SQLEnlight found that the Combo missed

- ✗ Live execution plan analysis (EX0018: 21% Sort cost in uspGetEmployeeManagers)
- ✗ SA0240: Stored procs don't return a result code — tracked across 53 objects
- ✗ SA0047: Specific un-indexed columns used in WHERE/JOIN (e.g., KEY_TBL.[KEY])
- ✗ SA0054: Parameter modified before use in query (affects query plan caching)
- ✗ SA0238: User-defined function in query filter causing performance problems
- ✗ SA0115: Variable assignment from SELECT with no rows — potential NULL bug
- ✗ SA0001: NULL constant compared with = instead of IS NULL
- ✗ EX0006: Missing FK constraints between Production tables
- ✗ SA0247B: MONEY/SMALLMONEY type — deprecated, use DECIMAL instead
- ✗ SA0133: GETDATE() called multiple times — should be stored in variable once
- ✗ Full object-by-object breakdown with line:column precision on every violation

Where both tools agreed (overlap)

<i>Type conv.</i>	Both found implicit type conversion issues (DATE vs DATETIME). TSQLAnalyzer flagged it schema-wide; SQLEnlight pinpointed it at line level.	Overlap
<i>No FK index</i>	Both flagged missing indexes on foreign key columns. SQLEnlight listed specific tables (SA0044); MS Assessment used EX0006 for missing FK constraints altogether.	Overlap
<i>Line length</i>	SQLFluff (LT05, 1 instance) and SQLEnlight (SA0261, 130 instances) both flagged long lines — showing the Combo only saw the tip of the iceberg.	Overlap
<i>Cluster ed idx</i>	TSQLAnalyzer (SRP0020) flagged HEAP tables; SQLEnlight found the same and also measured actual fragmentation of those heaps.	Overlap
<i>Trigger SELECT</i>	TSQLAnalyzer (SRP0004) and SQLEnlight (SA0013) both flagged triggers returning result sets.	Overlap

Verdict: Neither one alone is enough. The Combo gives you infrastructure health (disk, fragmentation, unused indexes, DB config) that SQLEnlight doesn't. SQLEnlight gives you deep semantic analysis with line-level precision and actual execution plan data that the Combo can't match. For a complete audit, you need both running together — which is exactly what your pipeline does.

Qualities That Push You to Use Each Tool

SQLFluff — use it when you care about code consistency and style enforcement

It auto-fixes problems, not just reports them: Most linters only tell you what's wrong. SQLFluff actually rewrites the SQL file and fixes formatting violations automatically. Run it in a CI pipeline and bad code never reaches the repo.

It works on files, not a live database: You don't need a running SQL Server. You give it a .sql file and it works offline. Perfect for a Git pre-commit hook — catch issues before code is even committed.

It supports multiple SQL dialects: T-SQL, PostgreSQL, MySQL, BigQuery, Snowflake, Spark SQL — all with one tool. Simple config change: dialect = tsql.

It enforces team standards through configuration: A .sqlfluff file lives in the repo. Every developer on the team gets the same rules applied automatically. No more 'it looked fine on my machine.'

It integrates natively into DevOps pipelines: pip install sqlfluff. Runs in GitHub Actions, GitLab CI, Azure DevOps, Jenkins. The exit code is non-zero on failure so the pipeline breaks if violations exist.

It catches formatting noise before code review: Things like inconsistent casing, long lines, empty GO batches — eliminated automatically so reviewers focus on logic, not style.

TSQLAnalyzer — use it when you care about T-SQL logic quality and best practices

It understands the semantics of your SQL, not just the syntax: Reads your logic and says 'this JOIN has no foreign key backing it' or 'this column exists with different types across schemas.' A completely different level of intelligence than a linter.

It enforces your company's naming standards through regex: Configure patterns like ^PK_Department\$ for primary keys or ^UX_Employee_.* for unique indexes. Every object in the database is checked automatically.

It catches silent data bugs: @@IDENTITY vs SCOPE_IDENTITY(), implicit type conversions, nullable columns in predicates — bugs that don't throw errors but produce wrong data silently.

It integrates Microsoft's own static analysis rules: The SR0007, SR0008, SR0014 rules are the same ones used by Visual Studio Database Projects at build time.

It gives you a scored quality report with severity tiers: The 0–100 score gives management a single number to track over time. Set a quality gate: no deployment if score is below 70.

It works without executing anything: Pure static analysis — no queries run, no execution plans generated. Fast, safe, works even on a database you don't have write access to.

Microsoft SQL Assessment — use it when you care about server and infrastructure health

It's the only tool that checks what's happening at runtime on the server itself: Doesn't read code — reads the live state of SQL Server using DMVs. No other tool knows that your disk has 1% free space or that indexes have never been used.

It finds infrastructure problems that code analysis can never see: Recovery model, Query Store status, DBCC CHECKDB history, data file free space — server-level settings that exist outside of any SQL code.

It detects unused indexes with real usage statistics: Reads sys.dm_db_index_usage_stats — the actual counter of how many times each index was used since the last SQL Server restart.

It measures actual index fragmentation: Not estimated — it runs sys.dm_db_index_physical_stats and gives you exact percentages like 23.71%. This is what you need to decide whether to REBUILD or REORGANIZE.

It's from Microsoft, so it always reflects current SQL Server best practices: The rulebook is maintained and updated by Microsoft. You're always aligned with Microsoft's own standards.

It produces a JSON output that feeds into Azure Monitor and Azure Data Studio: If you're in a Microsoft ecosystem, this connects directly into your monitoring and alerting infrastructure.

SQLEnlight — use it when you care about deep semantic analysis with live execution intelligence

It's the only tool that reads actual execution plans: Runs your queries with SET STATISTICS XML ON, captures the XML execution plan SQL Server produces, and analyzes it. When it says Sort costs 21%, that's what SQL Server measured. No other tool does this.

It gives line-level and column-level precision on every violation: Every finding comes with (line:column) coordinates — e.g., (20:24) for an implicit conversion. A developer can jump directly to the exact character causing the problem.

It catches 100% of analyzed objects having issues and gives per-object reports: The output groups violations by object — stored procedure by stored procedure, view by view, trigger by trigger. Easy to assign specific objects to specific developers.

It detects performance anti-patterns that only appear in complex logic: User-defined function in WHERE clause filter (SA0238), variable modified before use breaking plan caching (SA0054), GETDATE() called multiple times (SA0133).

It calculates technical debt in hours: The 214.25 hours figure gives project managers a concrete estimate of remediation time. A business metric, not just a technical one.

It measures cognitive and cyclomatic complexity: Cyclomatic complexity of 107 on a single object means 107 independent execution paths — untestable and unmaintainable. Critical for identifying stored procedures that need refactoring.

It analyzes the database as a whole system: Finds missing foreign key constraints between tables (EX0006), cross-table type inconsistencies, and schema-wide patterns only visible when looking at the entire database at once.

One-line summary for each tool

Tool	Reach for it when you need
SQLFluff	Style enforcement, auto-fix, Git/CI integration, multi-dialect support
TSQLAnalyzer	T-SQL logic quality, naming standards, silent bug detection

MS Assessment	Server health, disk, fragmentation, unused indexes, runtime state	
SQLEnlight	Execution plan analysis, line-level precision, complexity metrics, depth	full semantic

Does SQLEnlight Run DBCC Commands?

Short answer: No. SQLEnlight does not run DBCC commands.

SQLEnlight is a static and semantic analyzer. It reads your T-SQL code, resolves types, and captures execution plans — but it deliberately stays in read-only, non-invasive territory. DBCC commands like DBCC CHECKDB, DBCC SHOWCONTIG, or DBCC DBREINDEX are administrative commands that do heavy I/O work directly on database pages, locks, and physical storage. Running those automatically during an analysis scan would be dangerous on a production server — they can cause blocking, consume significant I/O, and in some cases acquire locks.

What SQLEnlight does instead for the things DBCC would cover:

<i>Fragmentation</i>	It queries sys.dm_db_index_physical_stats() — a DMV that reports fragmentation levels without the overhead or locking behavior of DBCC SHOWCONTIG. However, the fragmentation percentages in your output came from the Microsoft SQL Assessment, not SQLEnlight.	DMV
<i>Integrity</i>	SQLEnlight doesn't check physical page integrity at all. That's exclusively what DBCC CHECKDB does, and only the Microsoft SQL Assessment flagged the recommendation to run it.	Not covered
<i>Index stats</i>	SQLEnlight reads sys.dm_db_index_usage_stats and sys.indexes to understand index structure, but this is passive DMV querying, not DBCC.	DMV

Important: In your pipeline, the Microsoft SQL Assessment is the only tool that touches infrastructure-level concerns that DBCC normally covers — and even it only recommends you run DBCC CHECKDB, it doesn't run it itself either. Nobody in this pipeline actually executes DBCC commands. They are all suggesting you schedule them separately as a DBA maintenance job.

How Each Tool Works — From Scratch

SQLFluff

Think of it as ESLint but for SQL. It's a Python-based, open-source linter that reads SQL files and compares them to a ruleset.

Step 1 — Input: A plain .sql file (in your case, generated from the .dacpac).

Step 2 — Parsing: SQLFluff tokenizes and parses the SQL according to the dialect you configure (e.g., `tsql` for T-SQL). It builds an abstract syntax tree (AST).

Step 3 — Rule matching: Each rule (LT05, CP02, TQ03, etc.) is a Python class that inspects parts of the AST. Rules fire when a pattern is matched.

Step 4 — Auto-fix: Some rules can rewrite the AST and produce corrected SQL. Unfixable rules produce violations only.

Step 5 — Configuration: A `.sqlfluff` file in the repo root. You set `dialect = tsql`, enable/disable rules, set `max_line_length = 80`, etc. Rules can be excluded with `exclude_rules = LT01, CP01`.

TSQLAnalyzer

A static analysis engine for T-SQL. It parses your database schema and applies a library of best-practice rules without running anything against a live server.

Step 1 — Input: Connects to the SQL Server and reads all object definitions (stored procs, views, triggers, tables) from system catalogs like `sys.sql_modules`, `sys.objects`.

Step 2 — Parsing: Builds a T-SQL parse tree for each SQL module (proc, trigger, etc.) in memory.

Step 3 — Rule evaluation: Each rule (SRD*, SRN*, SRP*) is a visitor that walks the parse tree looking for patterns — e.g., 'is there a JOIN where no FK exists?'

Step 4 — Microsoft SQL Analysis integration: Invokes the `Microsoft.Build.Sql` static analysis rules (SR0007, SR0008, SR0014) — the same rules used by Visual Studio Database Projects during build time.

Step 5 — Configuration: A config file or command-line flags let you set severity thresholds, exclude rules, configure naming regex patterns (like `^PK_Department$` for SRN0007), and set acceptable complexity thresholds.

Microsoft SQL Assessment

Microsoft's official server-health assessment tool. Unlike the code analyzers, this one doesn't read T-SQL source — it interrogates the live SQL Server instance using DMVs.

Step 1 — Input: A live SQL Server connection. No source code needed.

Step 2 — Data collection: Runs queries against DMVs like `sys.dm_db_index_usage_stats` (index usage), `sys.dm_db_index_physical_stats` (fragmentation), `sys.databases` (recovery model, Query Store), and `sys.master_files` (disk space).

Step 3 — Rule evaluation: Compares collected data against a JSON-based rulebook (Microsoft-provided, updatable). Findings are categorized as Low/Medium/High.

Step 4 — Output: JSON report (`SQLAssessment_AdventureWorks2022.json`) and console output. Can integrate with Azure Monitor and Azure Data Studio.

Step 5 — Configuration: Invoked via PowerShell (`Invoke-SqlAssessment`). Customize rules using a JSON override file to tune thresholds (e.g., change the fragmentation alert from 10% to 20%).

SQLEnlight

The most sophisticated tool in this pipeline. It combines static analysis and live execution plan analysis, giving it the deepest insight of any tool here. Built by Yubitsoft.

Step 1 — Input: A live SQL Server connection + optional object name filter (in your case, * = entire database).

Step 2 — Schema extraction: Reads all object definitions from the database catalog, builds an internal object model with full type resolution (it knows that column X in table Y is INT and nullable).

Step 3 — Static analysis: Parses each stored proc, trigger, view, and function. Applies 200+ SA rules (SA0001 through SA0277+) that check logic, performance patterns, naming, and safety. Reports line:column precision.

Step 4 — Execution plan analysis: Actually executes queries with SET STATISTICS XML ON and captures the XML execution plan from SQL Server. Parses plan nodes (Sort, Hash Match, Index Scan) and flags high-cost operations (EX0018).

Step 5 — Configuration: Via the SQLEnlight GUI (desktop app) or command-line with a profile XML. Choose which rule groups to activate, set complexity thresholds, configure report templates (HTML, JSON, CSV), and define custom naming patterns.